

Calidad de Software

Ingeniería de Software 4

Luz Mery Rios Alzate | Ingeniería de Sistemas

Agenda

- Deuda Técnica
- Presentación del proyecto
- Pruebas de Software

Deuda Técnica en Proyectos de Software

2°

UCO
Universidad Católica de Oriente
Vigilada Mineducación

ALTA CALIDAD

Analizar, comprender y proponer estrategias de gestión de la deuda técnica en un proyecto de software, reconociendo sus causas, consecuencias y su impacto en la calidad y sostenibilidad del sistema.



"Decisiones de hoy que pueden convertirse en costos mañana."

Propósito de la actividad

Comprender, analizar y argumentar el impacto de la deuda técnica en proyectos de software, identificando sus causas, tipos, consecuencias y alternativas para su gestión, a partir del análisis de un caso real o simulado.

1 A. INTRODUCCIÓN

Comprender el problema

- ¿Qué es la deuda técnica?
- ¿Por qué se utiliza el término "deuda"?
- ¿Cómo surge en un proyecto de software?
- ¿Toda deuda técnica es negativa?
- ¿Por qué puede ser una decisión válida?
- Diferencia entre deuda intencional y no intencional.

Pregunta para iniciar:
¿Entregar una funcionalidad rápidamente, sabiendo que después será necesario corregirla, puede ser una decisión válida de ingeniería?

2 B. MARCO DE REFERENCIA

Comprender el concepto

- Definición de deuda técnica.
- Origen del concepto.
- Características principales.
- Tipos de deuda técnica:
  - Deuda de código
  - Deuda de diseño
  - Deuda arquitectónica
  - Deuda de pruebas
  - Deuda de documentación
  - Deuda de infraestructura
  - Deuda de procesos
  - Dependencias
- Causas más comunes.
- Relación con la calidad del software.

3 C. CASO DE ANÁLISIS

Llevar el concepto a una situación real

- Seleccionar o recibir un caso de software (real o simulado).
- Descripción del sistema y su contexto.
- Problema o situación actual del sistema.
- Antecedentes y decisiones que generaron la deuda técnica.



4 D. ANÁLISIS DEL CASO

Comprender las consecuencias

- Identificar la deuda técnica existente.
- ¿Qué decisiones generaron la deuda?
- Consecuencias para el sistema, el equipo y el negocio.
- Riesgos asociados.
- Costos actuales y futuros.
- ¿Qué pasa si no se hace nada?

5 E. PROPUESTA DE GESTIÓN

Tomar decisiones profesionales

- ¿Qué harían con la deuda técnica?
- Acciones de mejora (refactorizar, rediseñar, mejorar, automatizar, documentar, etc.).
- Justificación de las decisiones.
- Recursos necesarios.
- Beneficios esperados.
- Riesgos y alternativas.
- ¿Qué deuda mantener, reducir o eliminar?

6 F. CONCLUSIONES

Reflexionar y argumentar

- ¿Qué aprendieron sobre deuda técnica?
- ¿Por qué es importante reconocerla durante el desarrollo?
- ¿La deuda técnica siempre debe eliminarse?
- ¿Cómo afecta la sostenibilidad del sistema?
- ¿Qué papel tiene el ingeniero de software en su gestión?

7 G. PRODUCTO FINAL

Exposición y entrega

- Presentación de 15 minutos + 5 minutos de preguntas.
- Todos los integrantes deben participar.
- Utilizar diagramas, tablas, ejemplos y evidencias.
- Documento o presentación que incluya:
  - Caso analizado
  - Identificación de deuda
  - Análisis de consecuencias
  - Propuesta de gestión
  - Conclusiones
  - Referencias.

Trabajo
Individual o en equipo de máximo 4 estudiantes.

Tiempo de exposición
15 minutos + 5 minutos de preguntas.

Fuentes
Mínimo 2 referencias sobre deuda técnica y 1 artículo científico o trabajo académico. Citas en formato APA.

Criterios de evaluación
Comprensión del problema, análisis, argumentación, propuesta de gestión, calidad de la presentación y uso adecuado de fuentes.

¿Qué sigue?

Proyecto – Entrega de Implementación

Parte 1. Planeación



1. Planner del proyecto

Entrega 6 octubre:
Funcionalidad por estudiante terminada

Entrega 20 de octubre:
Corrección de los errores y entrega final del proyecto.



2. Organización del proyecto

Definir roles.

Crear el repositorio.

Establecer la estructura inicial del proyecto.

Definir reglas básicas para el trabajo colaborativo.



3. Ambiente de desarrollo

Crear el proyecto.

Configurar las herramientas y dependencias.

Configurar la base de datos.

Verificar la ejecución del proyecto.

Crear README con las instrucciones para ejecutar el proyecto.



Entregable

Planner.

Enlace al repositorio.

Documento breve de organización del proyecto.

Evidencias de configuración y ejecución del ambiente.

README del proyecto.

A la Verdad, por la fe y la ciencia

¿Qué sigue?

Proyecto – Entrega de Implementación

Parte 2

Funcionalidad implementada

Código integrado

Commit(s) identificables.

Código asociado a la funcionalidad.

Integración con la rama principal o mecanismo definido por el equipo.

Evidencia de funcionamiento

Capturas de pantalla.

Video corto.

Evidencia dentro del sistema funcionando.

Pruebas integración

Mínimo 5 casos de prueba

Entregable

Repositorio actualizado

Planner actualizado

Ficha de cada funcionalidad y Casos de prueba.

¿Qué sigue?
Proyecto – Entrega de Implementación
Parte 3

Una funcionalidad se considera terminada cuando:

- ❑Está implementada.
- ❑Está integrada al proyecto.
- ❑Puede ejecutarse.
- ❑Cumple el comportamiento esperado.
- ❑Tiene validaciones básicas.
- ❑Tiene datos de prueba.
- ❑Tiene mínimo cinco casos de prueba definidos.
- ❑Está disponible en el repositorio.
- ❑Puede ser demostrada por el estudiante.

UCO Universidad Católica de Oriente Vigilada Mineducación
Renovación Acreditación Institucional en ALTA CALIDAD
A la Verdad, por la fe y la ciencia

¿Qué sigue?

Proyecto – Prueba de la Funcionalidad Parte 3

Presentación de la funcionalidad desarrollada	Pruebas realizadas por el otro equipo	Entregable del equipo evaluador
Nombre de la funcionalidad. Propósito de la funcionalidad. Problema o necesidad que resuelve. Flujo de funcionamiento. Demostración de la funcionalidad en el sistema. Integración con las demás funcionalidades, cuando aplique.	Cada integrante debe realizar las pruebas de la funcionalidad asignada.	Presentación/demostración de la funcionalidad. Casos de prueba ejecutados. Matriz de resultados de las pruebas. Evidencias de las pruebas realizadas. Registro de errores o situaciones encontradas.

UCO
Universidad Católica de Oriente
Vigilada Mineducación

REPORTAR BUG

mantis
BUG TRACKER

¿Qué es Mantis Bug Tracker?

Herramienta Open Source

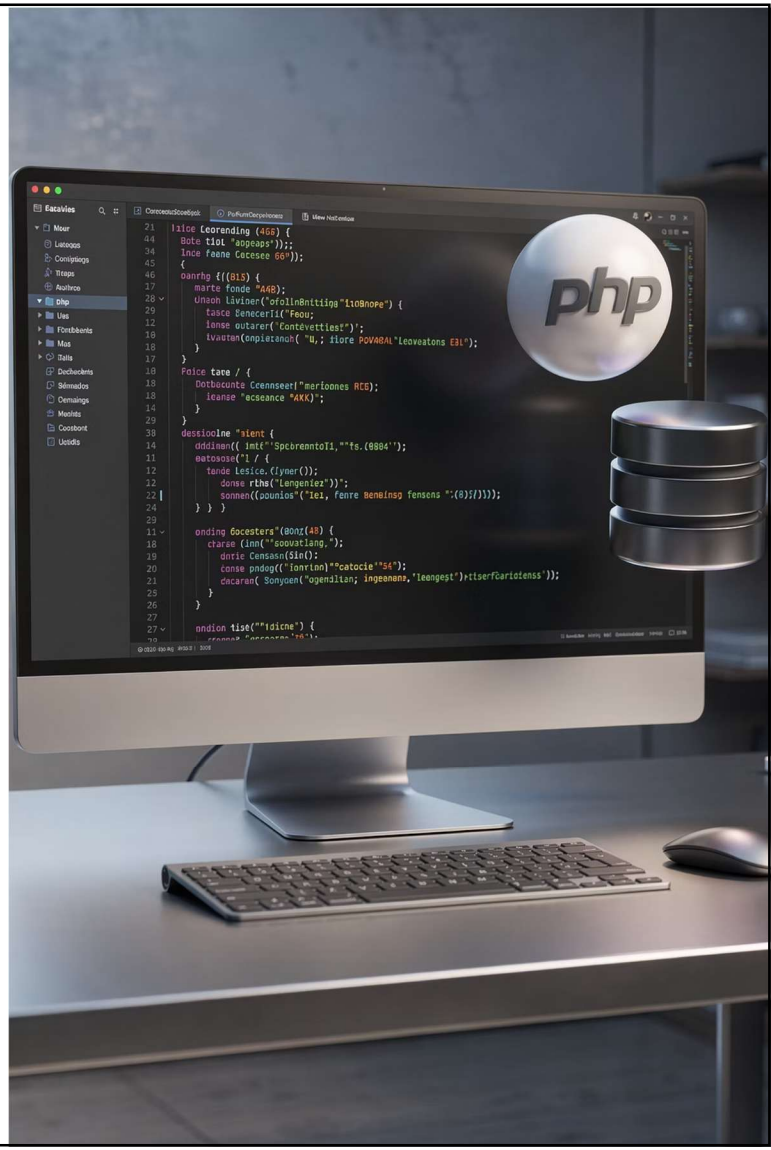
Plataforma gratuita para el seguimiento de errores (bugs) y gestión de tareas en proyectos de software.

Tecnología Accesible

Desarrollado en PHP y MySQL, destaca por su facilidad de instalación, configuración y personalización.

Ciclo de Vida Completo

Permite a equipos de desarrollo y QA gestionar eficientemente cada incidencia desde su reporte hasta su cierre.



El Tablero Inicial: **My View**

¿Qué es My View?

El punto de entrada personalizado de Mantis. Centraliza toda la información relevante para el usuario en una sola pantalla, sin necesidad de buscar en múltiples secciones.

→ **Vista Personalizada**

Muestra las incidencias asignadas al usuario, filtradas por estado activo.

→ **Resumen Rápido**

Visión instantánea del estado actual de tareas y prioridades pendientes.

→ **Navegación Sencilla**

Acceso directo a "Reportar Incidencia", "Ver Incidencias" y gestión de proyectos.

Reportar una Incidencia: Información Esencial

1

Categoría y Proyecto

Asigna la incidencia a la categoría correspondiente dentro del proyecto afectado.

2

Severidad y Prioridad

Define el impacto (*Menor, Mayor, Bloqueador*) y la urgencia de corrección (*Baja, Media, Urgente*).

3

Reproducibilidad

Indica con qué frecuencia ocurre el error: Siempre, A veces, Aleatorio o Nunca.

4

Entorno (Plataforma/SO)

Detalla el sistema operativo, navegador y versión donde se presenta el error.

Reportar una Incidencia: Detalles Cruciales



Descripción Detallada

Explica el fallo incluyendo el **comportamiento esperado vs. el actual**. Sé específico y objetivo.

Pasos para Reproducir

Secuencia clara y numerada de acciones que llevan al error. Sin ambigüedad.

Archivos Adjuntos

Sube capturas de pantalla, logs o archivos relevantes que ilustren el problema visualmente.

Referencias:

- Kaner, C., Bach, J., & Pettichord, B. (2002). Lessons Learned in Software Testing. Wiley. ISBN: 978-0471081128.
- Myers, G. J., Sandler, C., & Badgett, T. (2011). The Art of Software Testing (3rd ed.). Wiley. ISBN: 978-1118031964.
- Jorgensen, P. C. (2013). Software Testing: A Craftsman's Approach (4th ed.). Auerbach Publications. ISBN: 978-1466560680.
- Crispin, L., & Gregory, J. (2009). Agile Testing: A Practical Guide for Testers and Agile Teams. Addison-Wesley. ISBN: 978-0321534460.
- Graham, D., Black, R., & van Veenendaal, E. (2012). Foundations of Software Testing: ISTQB Certification (3rd ed.). Cengage Learning. ISBN: 978-1408044056.

Referencias:

- Whittaker, J. A. (2009). Exploratory Software Testing. Addison-Wesley. ISBN: 978-0321636416.
- Kinsbruner, E., Craske, A., & Feloney, S. (2019). Continuous Testing for DevOps Professionals. Apress. ISBN: 978-1484253505.
- Weinberg, G. M. (2008). Perfect Software and Other Illusions About Testing. Dorset House. ISBN: 978-0932633699.
- Whittaker, J. A. (2002). How to Break Software: A Practical Guide to Testing. Addison-Wesley. ISBN: 978-0201796193.
- Black, R. (2012). Advanced Software Testing - Vol. 1: Guide to the ISTQB Advanced Certification as an Advanced Test Analyst (2nd ed.). Rocky Nook. ISBN: 978-1937538497.

UCO
Universidad Católica de Oriente
Vigilada Mineducación

ALTA CALIDAD



¡Gracias!